第九章 Swing图形用户界面

1. Swing框架

- **Swing的两种元素：组件和容器**
- **Swing的组件继承于 `JComponent` 类**
- **容器的分类**
  - **重量级： `Jframe` ， `Jdialog` ，作为顶层容器**
  - **轻量级：作为中间容器**
- 布局管理器的作用
- Swing是否线程安全

2. Swing图形处理、绘制颜色的原理

- `Jframe` ： 顶层窗口，Jframe由用户的窗口系统绘制。
- 绘制2D图形：阅读代码

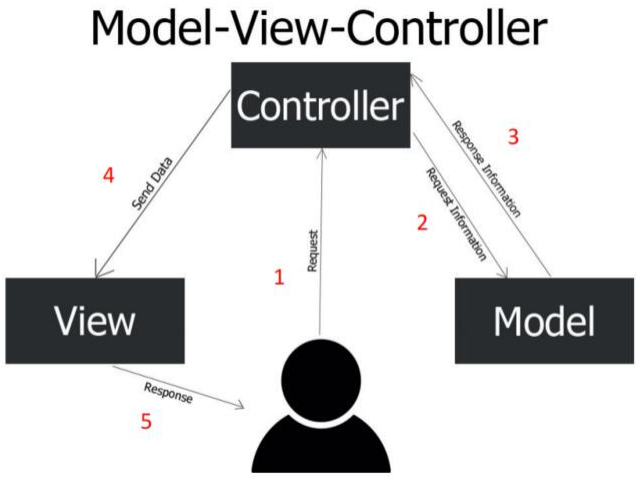
3. 事件处理机制

- **事件**：用户对组件的一次操作
- **事件源**：事件发生的场所，通常就是各个组件如按钮或滚动条
- **事件监听器**：实现了监听器接口的类实例

4. Swing基本用户组件

- 了解文本输入的三种方法（ `JTextComponent` 类）
  - 文本域
  - 文本区
  - 密码域
- 选择组件
  - 复选框 `JCheckBox`
  - 单选按钮 `ButtonGroup` ， `JRadioButton`
  - 组合框 `faceCombo`
  - 菜单 `JMenuBar`

5. MVC模式



- 概念
  - **模型（Model）**
  - **视图（View）**
  - **控制器（Controler）**
- MVC模式中，模型部分包含DAO层
- MVC中体现了单一职责原则
- Swing中利用MVC模式的方式

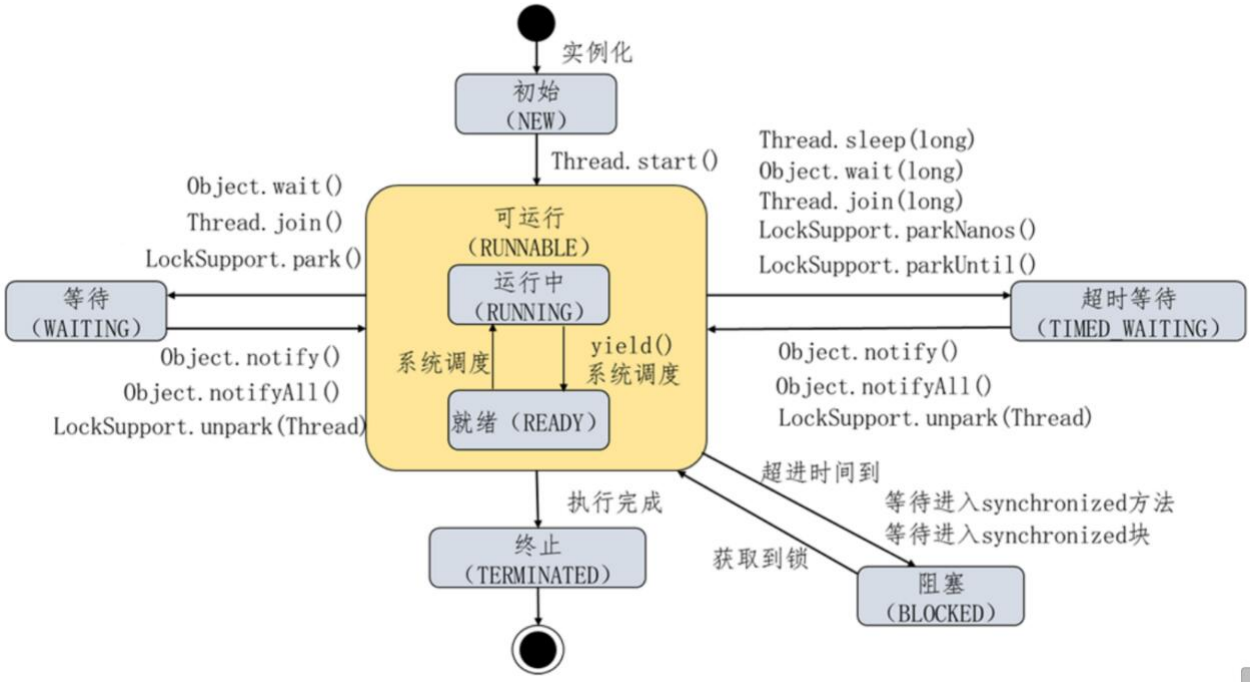
第十章 多线程

1. 进程与线程的区别

进程(Process)	**线程(Thread)**
重量级	轻量级
一个应用可以包含多个进程	一个进程可以包含多个线程
多个进程间不共享内存	一个进程的多个线程间共享内存
进程表现为虚拟机	线程表现为虚拟CPU
资源分配的最小单位	程序执行的最小单位

2. Java中对线程的控制

- 线程的状态（6种）



- **线程的两种建立方式**
  - `Thread` **子类，** `Runnable` **接口**
  - **为什么Runnable更常用**
- `Interrupt()` 用法
- 等待状态和计时等待状态的区别
- 守护线程的作用
- **区分Thread类和Object中的线程相关方法**

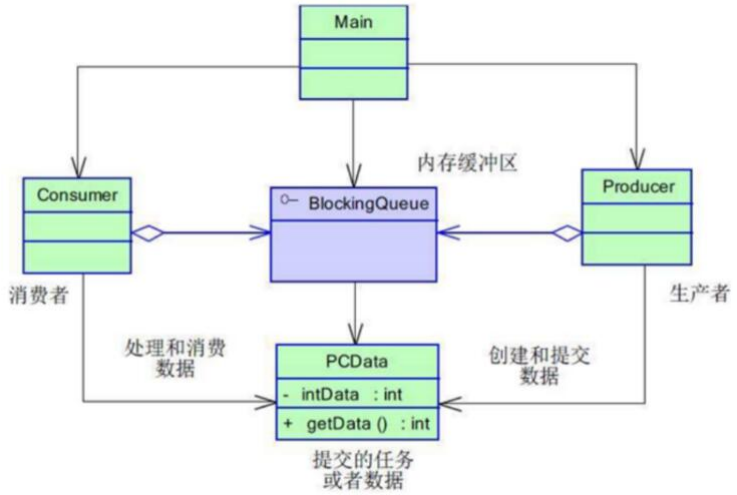
3. 同步、死锁及如何避免

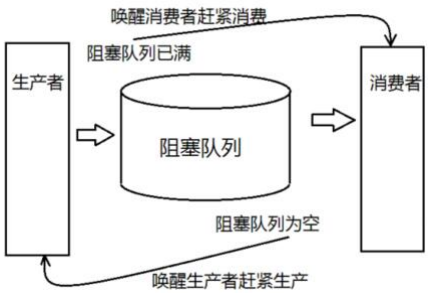
- 为什么需要**线程同步**？（原子性操作）
- 怎么**实现同步**：使用 `synchronized` 关键字对一个对象进行加锁
- 什么情况下会**产生死锁**

4. 任务与线程池

- 线程的另一种建立方式
  - `callable` **，有返回值，可抛出异常**
  - 要结合 `FutureTask` 使用
- 为什么要引入线程池?
  - **利用池化技术减少资源对象的创建次数**
  - **降低资源消耗**
  - **提高响应速度**
  - **提高线程的可管理性**
- 理解线程池的多任务控制方式

5. 多线程应用——生产者与消费者模式





- 生产者消费者设计模式的特点
- 如何简单地实现一个生产者消费者模式
  - 为什么缓冲区的判断条件是 `while` ， 而不是 `if` ？
  - 理解 `wait()` 方法为什么要放在同步块里?
- 生产者消费者设计模式的优点
  - 并发（异步）
  - 解耦